

Design the Interactions First

JOAT Labs, pm-substrate, and the Project-Manager Layer for Agentic Workspaces

By Emmanuel Akinwale, Founder, JOAT Labs

Executive Thesis

Modern businesses do not suffer from a shortage of software. They suffer from a shortage of coordination. The average organization already has specialized systems for sales, operations, finance, documents, communications, analytics, and delivery. Each tool may be competent inside its own boundary. The failure appears between those boundaries: state is copied, context is lost, ownership is unclear, and every new tool or AI feature adds another place where work can drift out of sync.

JOAT Labs is building for that gap. The company is not trying to add one more isolated application to the workspace. It is building the project-manager layer for the workspace: the layer that keeps tools, people, workflows, and agents aligned around shared state. In software terms, that layer is pm-substrate: a tenant-scoped operational substrate made of an entity graph, append-only event log, capability registry, workflow runtime, permission model, and continuity ledger.

The central claim is simple: if the value of a system comes from the interaction of its parts, then the modern workspace must be designed around interactions first. In the AI era, this becomes even more urgent. Agents can call tools, but tool access is not the same as operational memory. Agents need durable state, causality, permissions, workflow position, and a reliable model of what changed. Without that substrate, agents inherit the same fragmentation that already makes SaaS work expensive and brittle.

The Problem Is Not More Tools

Most business software has been built as if the unit of value were the individual tool. Better CRM. Better project tracker. Better calendar. Better inbox. Better accounting system. That orientation made sense when the market lacked capable components. It makes less sense now. Modern companies are surrounded by competent components, yet their operating reality is still fragmented.

The empirical picture is clear. BetterCloud's 2025 State of SaaS report finds that organizations manage an average of 106 SaaS tools. Zylo's 2024 SaaS Management Index reports that companies use only about half of their provisioned SaaS licenses and leave an average of \$18 million in annual license waste. MuleSoft's 2026 Connectivity Benchmark Report finds that 95% of organizations face integration challenges, while half of AI agents already operate in isolated silos. The market is not telling us that businesses need another disconnected surface. It is telling us that the surfaces no longer compose.

Recent human-computer interaction research makes the same point at the level of everyday work. Jahanlou and colleagues' 2023 study of task-centric application switching shows that knowledge workers often move across multiple applications to complete a single task, driven by tool limitations, workflow requirements, content movement, collaboration, and external constraints. Application switching is not merely a distraction problem. It is a symptom of work being distributed across systems that do not share enough context to behave like one environment.

This is the workspace problem JOAT Labs is organized around. Businesses do not need a new shelf. They need the shelf to become coherent.

The Hidden Bottleneck Is State

The word integration often makes the problem sound narrower than it is. Integration suggests that Tool A needs to send data to Tool B. That is necessary, but it is not sufficient. The harder question is whether the workspace has a reliable shared understanding of what the data means, who owns it, when it changed, which version is current, what process it belongs to, and what actions are allowed next.

That is a state problem. State is not just a database row. State is the current shape of the business: the customer and the contract, the project and its deadlines, the vendor and its obligations, the invoice and its approval status, the message and the event it caused, the workflow and the step it is waiting on. When state is scattered across apps, every tool builds a partial reality. Humans then become the integration layer, manually carrying context from one surface to another.

Enterprise information systems research has been circling this issue for decades, but recent work has sharpened it. Sunyaev and colleagues' 2023 essay on the future of enterprise information systems argues that organizations do not really want rigid ERP systems; they want digital, flexible, integrated business processes. The authors point toward composable architecture and business processes as first-class, directly executable entities. That framing is close to the JOAT Labs thesis: the workspace should not merely contain records and apps; it should contain the stateful processes through which work advances.

A workspace without shared state produces three predictable costs. First, context cost: people and systems spend energy rediscovering what is already known somewhere else. Second, coordination cost: handoffs require meetings, messages, reminders, and manual reconciliation. Third, trust cost: when two systems disagree, no one knows which one represents reality. Those are not edge cases. They are the daily texture of fragmented work.

Agents Make the State Problem Impossible to Ignore

AI agents expose the workspace problem because agents are asked to operate across tools. A model can reason over a prompt, call an API, and produce an output. But a model call is not itself the system of record. It does not automatically know which facts are authoritative, which facts are stale, what changed since the last run, which tool owns a field, which workflow step is active, or whether the requested action is permitted.

Recent agent research makes this explicit. Wang and colleagues' 2024 survey of LLM-based autonomous agents organizes the field around profile, memory, planning, and action. Li and colleagues' 2024 survey of LLM-based multi-agent systems similarly treats perception, self-action, mutual interaction, memory, and evolution as core system components. Hatalis and colleagues' Memory Matters paper argues that pairing LLMs with autonomy requires memory systems because short-term context windows cannot carry complex work over time. Wu and colleagues' StateFlow work goes further, proposing state-driven workflows around LLM calls and tools so complex tasks can be represented as explicit state machines rather than improvised prompt chains.

The implication for business software is direct. Agents do not only need tool access. They need operational state. Protocols and connectors can help an agent reach tools, but they do not by themselves decide what the business currently knows, what the next valid transition is, or how to preserve causality across time. Without a substrate, each agent reconstructs reality from scattered SaaS APIs, retrieved snippets, chat history, documents, and stale summaries. That makes agents brittle, hard to audit, and difficult to trust.

This is why JOAT Labs treats the agent state problem and the workspace interoperability problem as the same problem viewed from two angles. Workspaces have fragmented state across tools. Agents fail when asked to act across that fragmented state. The answer is not merely a bigger context window or a better chatbot. The answer is a shared operational substrate.

Tools Are Departments; pm-substrate Is the Project Manager

The simplest analogy is an organization. A CRM is the sales department. A project tracker is operations. A document system is knowledge management. An accounting tool is finance. A calendar is scheduling. Each department can be excellent and still fail as part of the whole if handoffs are unclear, facts are inconsistent, and nobody is responsible for the cross-functional state of the work.

Well-run organizations solve that problem with project management. A project manager does not replace the specialists. The project manager keeps the objective stable, translates across functions, tracks commitments, notices blocked handoffs, and maintains the shared picture of what is happening. Recent organizational research supports the mechanism behind this. Xie and colleagues' 2022 study in the Journal of Business Research links inter-team coordination to information elaboration and team performance, especially where teams have knowledge-integration capability. The value comes not from any one function knowing more in isolation, but from functions being able to combine what they know.

JOAT Labs applies that operating principle to software. The tools remain specialists. The project-manager layer keeps their state, events, permissions, and workflows coherent. That layer is pm-substrate.

What pm-substrate Is

pm-substrate is the implementation of the JOAT Labs thesis. It is not a single vertical application and it is not a point-to-point integration script. It is a shared operational substrate that lets tools and agents coordinate through declared contracts instead of private assumptions.

At its core, pm-substrate has six responsibilities:

Substrate part	Responsibility
Entity graph	Maintains stable business entities and typed relationships across tenants, profiles, and tools.
Append-only event log	Records what happened, when it happened, what caused it, and which downstream processes should react.
Capability registry	Lets tools declare what they read, write, emit, consume, and require, so composition is explicit.
Workflow runtime	Represents business processes as stateful invocation graphs rather than hidden app code.
Permissions and validation	Checks whether an action is allowed and whether its inputs match the declared contract before mutation.
Continuity ledger	Gives agents durable checkpoints, evidence, decisions, and unresolved work across sessions.

This architecture changes the role of integration. A capability provider does not need to know every other provider. It declares its contract to the substrate. The substrate records events, updates state, validates inputs, enforces boundaries, and advances workflows. That means a new provider can compose with existing providers through typed events and shared state instead of custom bilateral coordination.

The design is intentionally boring at the infrastructure level. The power is not in exotic infrastructure. The power is in making identity, events, permissions, workflow, and state explicit enough that independent tools and agents can safely compose.

A Concrete Flow

Consider a wedding-services business. A vendor contract is signed. In a traditional SaaS stack, that fact may live in a contracts tool, while the calendar, planner, budget, and communications systems each need to be updated separately. Humans often become the bridge.

In pm-substrate, the flow can be represented as a chain of stateful events and capability invocations:

- 1 The contracts capability emits a typed event: `wedding.contract.signed`.
- 2 The event log records the event with tenant, entity, causation, and provenance metadata.
- 3 The calendar capability subscribes to the event and confirms the vendor deliverable window.

- 4 The planner capability unlocks downstream tasks linked to that contract and deliverable.
- 5 The budget capability updates projected and actual spend through the graph relationship between contract, vendor, and budget category.
- 6 The communications capability schedules a reminder to the couple and vendor 72 hours before the deliverable.
- 7 The continuity ledger preserves what the agent or workflow concluded, what evidence supported it, and what remains unresolved.

No single tool owns that whole outcome. The outcome emerges because the tools coordinate through the substrate. That is the project-manager layer in software form.

Why This Is Not Just iPaaS or Workflow Automation

It is natural to compare pm-substrate to iPaaS platforms, workflow automation tools, or API orchestration products. Those systems are useful, but they usually begin from connections: when this happens in one app, do that in another app. pm-substrate begins from shared operational state.

The distinction matters. Point-to-point automation can move data, but it often does not answer deeper questions: Which entity is this really about? Which system owns this fact? Is the downstream action still valid? What changed since the trigger fired? Which permission boundary applies? What happens when a second tool acts concurrently? How do we replay the history? How does an agent know which memory is evidence and which memory is stale?

pm-substrate is designed to make those questions first-class. It treats tools as capability providers over a common graph and event history. It treats workflows as stateful processes. It treats permissions and schemas as runtime gates. It treats agent memory as evidence-backed continuity, not just retrieved text. That is the difference between connecting apps and coordinating work.

The Research Foundation

The philosophical foundation is not new. Ludwig von Bertalanffy's general systems theory, Russell Ackoff's work on systems thinking, Donella Meadows' work on feedback structures, and Jay Forrester's system dynamics all argue in different ways that system behavior is produced by relationships and feedback loops, not by isolated parts. Sociotechnical systems theory, beginning with Trist and Bamforth, adds that work systems must jointly optimize social and technical components rather than treating technology as an isolated mechanism.

What has changed is the urgency and the surface area. The modern workspace is now a live system-of-systems: SaaS tools, documents, APIs, human teams, AI agents, automations, and external providers all interacting over time. Fisher's work on systems-of-systems interoperability warned that emergent behavior cannot be localized to a single component. Recent enterprise systems research, HCI research on application switching, and LLM-agent research on memory

and state all converge on the same point: coordination infrastructure matters.

JOAT Labs is building from that convergence. Systems theory explains why the interaction layer matters. Enterprise interoperability research explains why data, process, service, and organizational boundaries all matter. HCI research explains how fragmentation shows up in daily work. Agent research explains why memory, state, action, and interaction must be engineered rather than improvised. pm-substrate is the architectural response.

Why JOAT Labs

The name JOAT Labs is deliberate. JOAT points to the breadth of capabilities a modern business needs. Labs points to the place where those capabilities are fused into a coherent system. The company is not claiming that breadth alone is enough. The point is not to be a jack of all trades in the sense of scattered competence. The point is to make many specialized capabilities behave as one coordinated operating environment.

My own lens comes from industrial engineering, manufacturing engineering, and automation engineering. Those disciplines train you to look at flows, queues, handoffs, constraints, failure modes, and feedback. They also train you to notice when local optimization makes the whole system worse. That is the pattern in modern workspace software: every tool optimizes its own surface, while the business pays for the gaps between surfaces.

JOAT Labs exists to close those gaps. The project-manager layer is not a slogan. It is the state, event, permission, workflow, and continuity substrate that makes coordinated work possible.

Conclusion: Design the Interactions First

The next era of workspace software will not be won by adding isolated AI features to isolated tools. That path multiplies fragmented state. The next era will be won by systems that make tools and agents interoperable at the level where work actually happens: entities, events, permissions, workflows, memory, and decisions.

The system is the strategy because the value is not locked inside any one component. It appears when the components coordinate. It appears when a contract signing can update a calendar, unlock a task, adjust a budget, trigger a reminder, and leave an audit trail without a human carrying state across five apps. It appears when an agent can act with durable memory, current context, permission boundaries, and a workflow position rather than guessing from a prompt.

That is the purpose of pm-substrate. It is JOAT Labs' implementation of the project-manager layer for the modern workspace. Design the interactions first.

References

- Ackoff, R. L. (1994). On Systems Thinking. W. Edwards Deming Institute. <https://deming.org/ackoff-on-systems-thinking-and-management/>
- BetterCloud. (2025). State of SaaS 2025. <https://pages.bettercloud.com/rs/719-KZY-706/images/BetterCloud-State-of-SaaS-2025.pdf>
- Bertalanffy, L. von. (1968). General System Theory: Foundations, Development, Applications. George Braziller.
- Fisher, D. A. (2006). An Emergent Perspective on Interoperation in Systems of Systems. Software Engineering Institute, Carnegie Mellon University. https://www.sei.cmu.edu/documents/763/2006_005_001_14759.pdf
- Hatalis, K., Christou, D., Myers, J., Jones, S., Lambert, K., Amos-Binks, A., Dannenhauer, Z., & Dannenhauer, D. (2023). Memory Matters: The Need to Improve Long-Term Memory in LLM-Agents. AAAI Fall Symposium Series. <https://ojs.aaai.org/index.php/AAAI-SS/article/view/27688>
- Jahanlou, A., Vermeulen, J., Grossman, T., Chilana, P. K., Fitzmaurice, G., & Matejka, J. (2023). Task-Centric Application Switching: How and Why Knowledge Workers Switch Software Applications for a Single Task. Graphics Interface 2023. <https://www.research.autodesk.com/publications/task-centric-application-switching/>
- Li, X., Wang, S., Zeng, S., Wu, Y., Yang, Y., et al. (2024). A survey on LLM-based multi-agent systems: workflow, infrastructure, and challenges. Vicinagearth. <https://link.springer.com/article/10.1007/s44336-024-00009-2>
- MuleSoft. (2026). Connectivity Benchmark Report. <https://www.mulesoft.com/lp/reports/connectivity-benchmark>
- Sunyaev, A., Dehling, T., Strahringer, S., Xu, L. D., Heinig, M., Perscheid, M., Alt, R., & Rossi, M. (2023). The Future of Enterprise Information Systems. Business & Information Systems Engineering, 65, 731-751. <https://link.springer.com/article/10.1007/s12599-023-00839-2>
- Trist, E. L., & Bamforth, K. W. (1951). Some Social and Psychological Consequences of the Longwall Method of Coal-Getting. Human Relations, 4(1), 3-38.
- Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W. X., Wei, Z., & Wen, J. (2024). A survey on large language model based autonomous agents. Frontiers of Computer Science, 18, 186345. <https://link.springer.com/article/10.1007/s11704-024-40231-1>
- Wu, Y., Yue, X., Zhang, S., Cheng, C., Wang, H., & Zhang, D. (2024). StateFlow: Enhancing LLM Task-Solving through State-Driven Workflows. COLM 2024. <https://openreview.net/forum?id=3nTbuygoop>
- Xie, X., Ling, C.-D., Liu, W., & Wei, J. (2022). Inter-team coordination, information elaboration, and performance in teams: The moderating effect of knowledge integration capability. Journal of Business Research, 149, 149-160. <https://doi.org/10.1016/j.jbusres.2022.05.002>
- Zylo. (2024). SaaS Management Index. <https://zylo.com/news/2024-saas-management-index/>